Reproducing SimpleMem and Evaluating Retrieval Reflection

Abstract

SimpleMem retrieves memories from past conversations before answering a question. Its reflection step checks the evidence and searches again if the model judges it incomplete. We reproduced SimpleMem's LoCoMo workflow with DeepSeek and measured the effect of disabling reflection. Across three paired rounds on 120 fixed questions, macro token F1 was **42.33** with reflection and **42.57** without it. The enabled-minus-disabled difference was -0.23 percentage points and changed sign across rounds. Disabling reflection reduced mean question latency by **42.8%** and API token usage by **66.7%**. Both reductions occurred in all three rounds.

1 Project and reproduction target

A question about which country Tim visited most frequently asks an agent to connect records from several trips discussed across sessions. SimpleMem first turns the dialogue into searchable memory entries. It then chooses what to retrieve for a specific question and passes that evidence to an answer generator. Its combination of memory, planning and search makes it a multi-step agentic system [1]. LoCoMo provides long conversations and reference answers for evaluating this behavior [2].

What this study reproduces

The reproduction target is the upstream LoCoMo workflow from reading the dialogue and building its memory index to retrieving evidence, generating answers and scoring them. The paper reports 43.24 average F1 for SimpleMem with GPT-4.1-mini in Table 1 [1]. We ran the pipeline with DeepSeek on a balanced subset of three conversations and report the scores alongside the paper's result in Table 3.

What the experiment changes

The upstream default enables retrieval reflection. We tested whether its extra checks and searches recover useful evidence often enough to improve answers. The ablation disables those steps after initial retrieval. The two conditions use the same question set, memory snapshots, model configuration, answer prompt templates and scoring procedure. We measure macro token F1, latency, HTTP requests and token usage.

Code and materials · Repository and run guide · Frozen question manifest · Machine-readable results

2 How SimpleMem answers a question

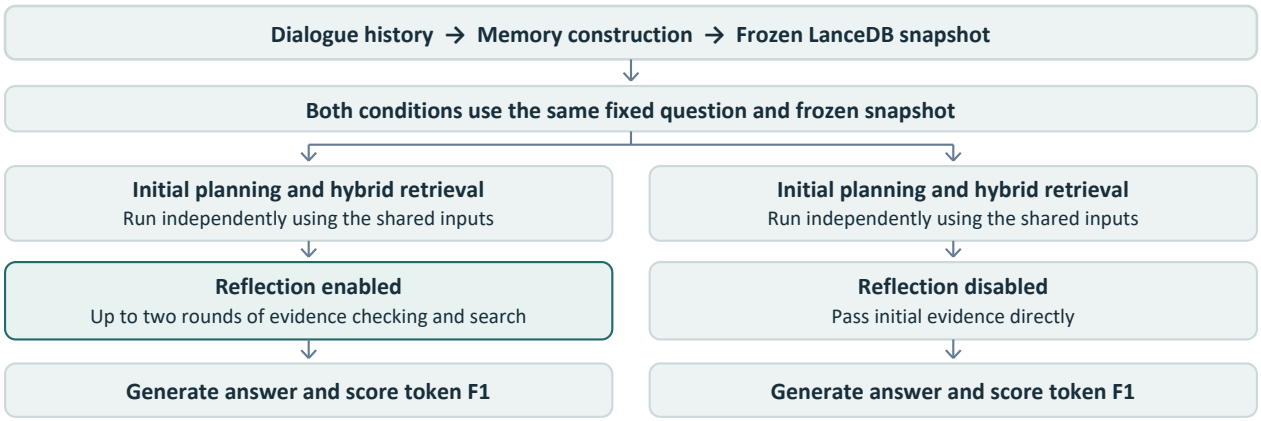
Writing searchable memories

Before the evaluation starts, the upstream system reads each conversation in windows and generates structured memory entries with metadata. The paper calls the consolidation of related facts during memory writing **Online Semantic Synthesis** [1, §2.2]. LanceDB stores the entries and their embeddings for similarity search, while a Tantivy index supports keyword search. Both conditions start from the same stored entries for every question about a conversation.

Planning and retrieving initial evidence

Given a question, the model identifies information it needs and generates targeted queries. The retriever combines three search signals. Semantic search matches meaning through embedding similarity, lexical search matches keywords, and structured search uses metadata such as people or time. The retriever merges the results and removes duplicates. Both conditions run these steps before the reflection switch takes effect.

Figure 1. SimpleMem workflow with independent retrieval in each condition



What reflection adds

The enabled branch asks the language model whether the current evidence is complete for the question. If it identifies missing information, it generates supplemental queries, searches again and merges additional results. The loop runs for at most two rounds. The disabled branch sends its initial evidence straight to the answer generator. Each branch produces a separate answer, which is scored against the same reference using the same token F1 implementation.

Each branch plans and searches independently. The initial lists had a mean Jaccard similarity of 0.85 (shared entry IDs divided by all distinct IDs in each pair). In 96 of 360 pairs, the lists matched in both entries and order. Section 7 traces two examples.

3 Environment and reproduction procedure

Table 1. Execution environment and controlled settings

Component	Configuration
Upstream source	SimpleMem v0.1.0, commit **7da777f**. The runner uses the upstream core code, prompt templates and LoCoMo scoring function.
Compute	Windows 11, Python 3.12.14 and PyTorch 2.8.0 CPU. Four Torch threads for local embeddings and three question workers per evaluation group.
Data and embeddings	LoCoMo JSON at cbfbc1db and Qwen3-Embedding-0.6B at 97b0c614. The runner checks the data file and model weights by SHA-256.
Index and memory window	LanceDB 0.25.3 with dense vector search and a Tantivy keyword index. The local memory window is 40 turns.
Hosted model and credentials	DeepSeek deepseek-flash via https://api.deepseek.com, with thinking disabled. The API key is supplied as DEEPSEEK_API_KEY in the environment or in a local ignored file.
Decoding	Temperature 0.2 for information planning, 0.3 for initial and supplemental query generation, and 0.1 for query analysis, completeness checks and answers. The client leaves top_p and max_tokens at provider defaults.

From a preliminary run to the formal experiment

The preliminary check built a memory database from the 419 dialogue turns in conv-26. It stored 460 entries, returned five hits on a known-term keyword probe, and answered and scored 12 questions. Its macro F1 was 42.22. This check verified ingestion, indexing, retrieval and scoring before the larger evaluation.

A separate paired pilot then evaluated those 12 questions under both reflection settings. All 24 pilot executions passed checks for scorable answers, retrieval traces and API usage records. The enabled branch exercised evidence checking and supplemental search. The disabled branch recorded neither operation. The pilot questions were excluded from the formal question set, and the verified conv-26 memory database was reused. The other two conversations each received their own frozen database before formal evaluation.

How to rerun the study

The repository run guide gives the complete Windows PowerShell commands. Install the pinned Python dependencies and CPU PyTorch in an isolated environment, download the pinned dataset and embedding model, and configure the local key. Run the experiment with **--mode all**, which checks the pilot before the three formal rounds. Then run **summarize.py** on the output directory to regenerate the aggregate metrics. The expected outputs and the checks that support them are described in Section 8.

4 Paired design and measurement

We used seed 5660 to select ten questions per category in each conversation. Single-hop questions focus on individual facts, multi-hop questions combine information, temporal questions focus on timing or order, and open-domain questions can draw on knowledge beyond the dialogue. We excluded the preliminary questions and category 5 adversarial items. Hosted model generation remained stochastic.

Table 2. Frozen inputs for the formal experiment

Conversation	Dialogue turns	Memory entries	Questions per round and condition
conv-26	419	460	40
conv-42	629	634	40
conv-43	680	685	40
Total	1,728	1,779	120

A pair is the same question in the same round under both settings. Each round yields 120 pairs, and three rounds yield 360 pairs and 720 answers. The runner alternates which condition goes first across conversations and rounds to balance run order. It checks database fingerprints before and after each group. The summarizer checks that each pair uses matching questions and snapshots and that disabled runs recorded no reflection checks or supplemental queries.

The runner creates a system against the existing database. The variable **reflection** is True in the baseline and False in the ablation.

```
system = benchmark.SimpleMemSystem(
    db_path=str(database), clear_db=False,
    enable_planning=True, enable_reflection=reflection
)
```

Reading the metrics

Token F1. The upstream scorer lowercases the text and compares sets of unique tokens. Precision is the fraction of predicted tokens found in the reference, and recall is the fraction of reference tokens recovered. Their harmonic mean is F1. For “The UK” against “UK”, precision is 1/2 and recall is 1, giving F1 = 2/3. We weight all four category means equally and report macro F1 on a 0-100 scale. Unscorable answers receive zero.

Latency and API usage. Per-question latency spans retrieval through final answer generation. HTTP attempts include retries. Token counts sum prompt and completion tokens reported by the API. These measures exclude memory construction and pilot runs. Differences are enabled minus disabled. Percentage reductions use the enabled condition as the baseline.

5 Reproduction and answer quality

Table 3. Published and local token F1 on a 0-100 scale

Category	Paper [1]	Enabled	Disabled	Enabled minus disabled
Multi-hop	43.46	38.76	37.99	+0.77
Temporal	58.62	53.11	53.21	-0.10
Open-domain	19.76	12.30	12.24	+0.06
Single-hop	51.12	65.16	66.81	-1.66
Macro F1	**43.24**	**42.33**	**42.57**	**-0.23**

Differences are calculated before rounding the displayed scores.

Reading the comparison with the paper

The published result is 43.24 macro F1 with GPT-4.1-mini on LoCoMo-10 [1, Table 1]. Our reflection-enabled baseline reached 42.33, a gap of 0.91 points. We ran the upstream workflow with DeepSeek on 120 questions drawn from three conversations and used a 40-turn window to build memories. The paper reports a 20-turn window and LanceDB 0.4.5 with an IVF-PQ vector index [1, Table 7]. Our snapshots used LanceDB 0.25.3 with a Tantivy keyword index, and the retriever searched the stored vectors directly. The model and window can change which facts are stored, and the search setup can change which entries are retrieved. The smaller question sample changes which answers contribute to F1. These differences occurred together, so the comparison cannot isolate any one of them.

The overall scores are close partly because the category gaps offset one another. Compared with the paper, our enabled baseline was about 14.04 points higher on single-hop questions and 7.46 points lower on open-domain questions. It also scored 4.70 points lower on multi-hop and 5.51 points lower on temporal questions. We reproduced the pipeline from dialogue ingestion to scored answers and obtained a similar overall score, with a different profile across question types.

How the two ablations differ

The authors describe replacing intent-aware retrieval planning with fixed-depth retrieval [1, §3.4]. Macro F1 fell from 43.24 to 37.78, a 5.46-point drop [1, Table 5]. Our experiment keeps intent-aware planning and initial hybrid search active in both conditions. The reflection switch controls the subsequent completeness check and supplemental search.

Each local category has 30 distinct questions, with ten from each conversation. The same questions appear in three rounds, yielding 90 paired observations per category. We average the four category scores equally. Use the paper column as a benchmark reference and the paired local columns to assess the reflection switch.

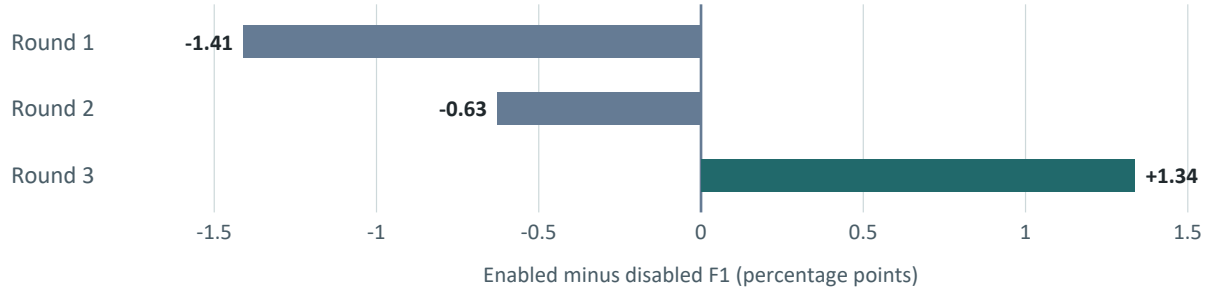
Answer quality across rounds

Mean paired difference -0.23 percentage points (pp). Sample variance of the three differences was 2.00 pp^2. Differences use unrounded scores.

Table 4. Variation across the three paired rounds

Round	Enabled F1	Disabled F1	Enabled minus disabled (pp)
1	41.31	42.72	-1.41
2	42.00	42.63	-0.63
3	43.69	42.35	+1.34
Sample SD	1.22	0.19	1.41

Figure 2. The F1 difference changed sign across rounds



How much weight to give the mean difference

The mean difference of -0.23 points is small relative to the 1.41-point standard deviation across rounds. The three trials showed no consistent F1 gain on the fixed questions. Each branch sampled its own planning and answer calls, so the spread includes variation in both evidence and answers. The repeated trials measure variation on the same questions and leave the effect on new conversations open.

Where the average hides opposing changes

Multi-hop F1 rose by 0.77 points overall. It fell by 3.24 in conv-26 and rose by 5.16 in conv-42. Single-hop F1 fell by 1.66 points overall, with a 2.47-point gain in conv-26 and a 4.05-point drop in conv-43. A second search can help when the first misses a fact needed to connect events, or draw an answer toward a related detail when the requested fact is already present. The variation across conversations makes question type alone a weak guide for deciding when to search again. Section 7 traces both mechanisms.

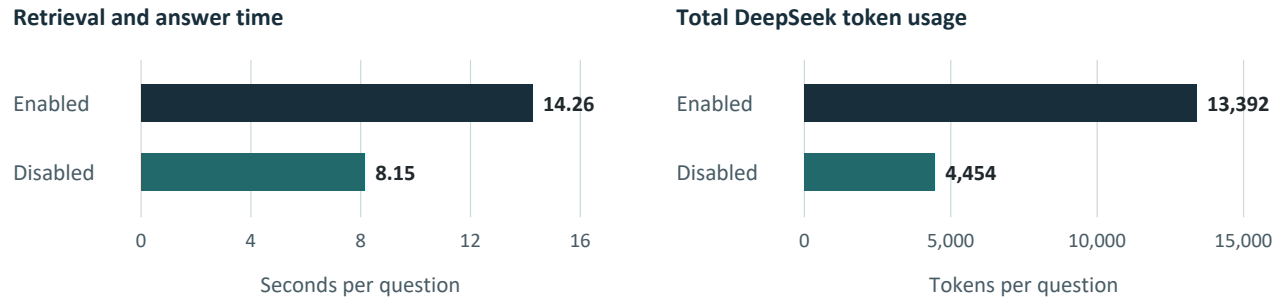
6 Latency and API usage

Table 5. Question-time latency and API usage

Mean per question	Enabled	Disabled	Enabled minus disabled
Retrieval and answer latency (s)	14.26	8.15	+6.10
Retrieval latency (s)	12.26	6.27	+5.99
Answer generation latency (s)	2.00	1.89	+0.12
DeepSeek HTTP attempts	6.47	4.00	+2.47
Total DeepSeek tokens	13,392	4,454	+8,938
Final answer prompt tokens	4,263	3,237	+1,026

All 720 answers were scorable and all 3,768 HTTP attempts had usage records. Differences use unrounded values. The paper's retrieval-efficiency token column uses a different scope from the sum of all API calls measured here.

Figure 3. Per-question latency and API token usage



Relative to the enabled baseline, disabling reflection reduced average latency by 42.8% and tokens by 66.7%. The reductions occurred in all three rounds. Enabled latency ranged from 13.87 to 14.62 seconds per question and disabled latency from 8.13 to 8.17 seconds. The corresponding token ranges were 13,197 to 13,527 and 4,438 to 4,462.

Where the overhead came from

Completeness checks averaged 4,632 tokens per enabled question, and generating supplemental queries averaged 3,280. Together these calls account for about 89% of the 8,938 additional tokens. The final answer prompt was also longer by about 1,026 tokens. This last number is already included in the total. Initial retrieval calls used about 1,150 tokens in either condition, so the largest change came after initial retrieval.

Retrieval accounted for 5.99 seconds of the 6.10-second increase. The extra work happens before answer generation, during evidence checks, supplemental query generation and search. Answer generation itself added 0.12 seconds on average.

The cost of checking when no entries are added

We selected the 181 pairs whose enabled run recorded an empty list of added memory IDs. For each pair, we subtracted disabled usage and latency from enabled usage and latency, then averaged the differences. The enabled run used 2,581 more tokens and took 2.28 seconds longer on average. The completeness check itself consumes resources even when it adds no entries.

7 What supplemental retrieval changed

The enabled runs produced 539 completeness decisions, including 349 judgments of incomplete evidence. A run can check completeness twice. Supplemental search added entries in 179 of 360 enabled runs (49.7%). Within those 179 pairs, the enabled answer scored higher in 33, lower in 36 and the same in 110. An added entry is a record that was absent from the initial list. Its usefulness depends on whether it supplies a fact the answer needs. The two cases below come from conv-43 in round 3.

Case A · Two separate trips support the UK answer

Question and initial evidence. The question asks which country Tim visited most often. The reference is “UK”. Both branches retrieved an entry about Tim reading about UK castles and wanting to visit them. The enabled branch also retrieved a record of Tim actually visiting a castle on a UK trip (D18:1). The disabled branch lacked that record and answered that it could not determine the country from the available context (F1 0.00).

What the extra search contributed. The enabled branch added 16 entries, including a record of Tim attending a Harry Potter conference in the UK on an earlier trip (D13:1). The two accounts gave the model evidence of repeated visits. It answered “The UK” (F1 0.67 against the one-word reference). The conference record shows how supplemental search can recover a missing fact. Because initial retrieval had already supplied one of the two supporting records, this pair cannot tell us how much of the F1 gain came from the extra search alone.

Case B · Edinburgh was a suggestion, with the destination still open

Question and initial evidence. The question asks where John and his teammates are planning to explore. Both branches started with the same 31 entries in the same order, including John's statement that the team planned to explore “a new city” (D11:7). The disabled branch used that phrase and said the team was still deciding. It matched all three reference tokens out of ten unique tokens in its answer (F1 0.46).

What the extra search contributed. The enabled branch added 39 entries, including later turns in the same session where Tim suggested Edinburgh and John said it was worth considering (D11:10-11). The answer reported Edinburgh as a possibility and kept the destination undecided. Its wording shared no tokens with “a new city” (F1 0.00). The extra evidence supported the suggestion, while the reference preserved the more general wording of John's original plan. This is a drop in reference overlap even though the named city was grounded in the dialogue. The enabled run used 19,856 tokens versus 3,831 with reflection disabled.

8 Debug diary and reproduction checks

Installation · LanceDB stopped before evaluation

The first run failed while importing LanceDB 0.25.3. The traceback named a missing **CreateEmptyTableRequest** in the namespace client, which pointed to a dependency version mismatch before any benchmark questions ran. We pinned both **lance-namespace** and **lance-namespace-urllib3-client** to 0.0.21. The next run passed the import and reached index construction. Both pins are recorded in the experiment requirements so the installation resolves the same versions on a fresh run.

Memory setup · The keyword index failed after memory writes

The next run logged a Windows permission error while Tantivy wrote a **.fieldnorm** file and skipped full-text index creation. Storing memories in LanceDB was therefore only a partial setup check. We reran in a fresh database directory, confirmed that Tantivy created its index, and searched for a term known to occur in the stored memories. The keyword probe returned five matches. The 12-question preliminary run then verified that retrieval could pass evidence to the model and produce scored answers.

Formal evaluation · One query failed in the keyword channel

During the full run, Tantivy returned a syntax error for the keyword query for conv-43, question index 5. It happened in both settings in all three rounds. The upstream keyword handler returned an empty list for the rejected query, so the other retrieval channels still supplied evidence and all six answers were scored. We kept the upstream parser behavior and retained those answers. This was a per-query fallback after the index had been built, distinct from the earlier index write failure.

Checks that made the final results auditable

Before the formal run, the 24 pilot executions (12 pairs) verified scorable answers, retrieval traces and API usage. The enabled pilot exercised supplemental search and the disabled pilot recorded zero reflection checks and supplemental queries. In the formal run, the summarizer verified 360 matched pairs across three rounds. It checked question fields, snapshot fingerprints and reflection traces across the two branches. These checks confirm that the comparison uses the same stored inputs and that the switch affected the intended stage.

A rerun can inspect the question manifest and snapshot metadata first, then use the per-question records to check the initial and added memory IDs, answers and API usage. Those records are the basis for tracing both cases in Section 7.

Published aggregate summary · Summarizer implementation

9 Interpretation, limitations and conclusion

Why retrieving more produced uneven gains

Of the 179 pairs where supplemental search added entries, 110 (61.5%) tied on F1. An entry can be new to the retrieved list while repeating facts the answer already had or adding detail that leaves the scored wording unchanged. The case traces show why the value of an entry depends on what the question asks and how the model uses it.

What the paired comparison can explain

The comparison measures the effect of enabling the loop in the running system. The branches draw from the same stored entries and generate their own initial queries. Holding initial lists and their order fixed in a follow-up would isolate the effect of the extra evidence more closely. Reviewing answers against the supporting dialogue would help distinguish a factual error from a change in wording or specificity.

What carries over to a new run

The pinned dataset, embedding weights and runner let a new run repeat the evaluation procedure. The hosted model can write different memories when a database is rebuilt and produce different queries and answers even on an existing database. We controlled the first source of variation within each pair by sharing the snapshot. The three rounds describe repeatability on the same 120 questions, and measured latency also depends on API response times and the three-worker configuration.

Conclusion

We reproduced the end-to-end SimpleMem workflow with DeepSeek. The upstream code built searchable memories from LoCoMo dialogues, retrieved evidence, generated answers and scored all 720 responses. Across three rounds, both settings used the same 120 questions and frozen memory snapshots, so the reflection switch could be evaluated on matched inputs.

The paper reports **43.24** macro token F1, 0.91 points above our reflection-enabled baseline [1, Table 1]. The published score remains a reference point because our run used DeepSeek in place of GPT-4.1-mini, a three-conversation subset of LoCoMo-10, a 40-turn memory window in place of 20 turns, and a different index setup [1, Table 7]. These choices affect which facts are stored, which evidence is retrieved and which questions count toward F1. Their individual effects were not measured.

In the paired comparison, macro token F1 was **42.33** with reflection and **42.57** without it. The enabled-minus-disabled differences were -1.41, -0.63 and +1.34 points. Disabling reflection cut mean latency by **42.8%** and API token usage by **66.7%**, with reductions in every round. Evidence checks and supplemental query generation accounted for most of the extra tokens. Under the evaluated configuration, retrieval reflection did not demonstrate a consistent F1 improvement despite substantially increasing latency and token consumption.

References and reproduction materials

[1] Liu, J., et al. (2026). SimpleMem: Efficient Lifelong Memory for LLM Agents. ICML 2026 (PMLR 306). Table and section references use arXiv:2601.02553v3.

[2] Maharana, A., Lee, D.-H., Tulyakov, S., Bansal, M., Barbieri, F., & Fang, Y. (2024). Evaluating Very Long-Term Conversational Memory of LLM Agents. Proceedings of ACL 2024.

[3] AIMING Lab. SimpleMem v0.1.0, commit 7da777f. Upstream GitHub repository.

[4] SimpleMem Reproducibility Work (2026). Coursework repository. Comparison with the upstream baseline.